

UNIT III: MongoDB

- **Q1: Differentiate between SQL and NoSQL databases. Why do we use MongoDB?**
- **Q2 (High Priority): How do you connect MongoDB with Node.js? Write the code using a driver (or Mongoose).**
- **Q3: Explain MongoDB Data Types and the concept of Collections vs Documents.**
- **Q4: Write the syntax for CRUD Operations in MongoDB (`insertOne`, `find`, `updateOne`, `deleteOne`).**

SQL Databases

Structured Query Language

Relational model (tables with rows & columns)

Fixed / predefined schema

Vertical scaling

Uses SQL for queries

Stores data in tables

Best for complex queries and transactions

Strong ACID properties

Examples: MySQL, Oracle, PostgreSQL

Slower for huge unstructured data

NoSQL Databases

Not Only SQL

Non-relational model (Document, Key-value, Graph, Column-family)

Dynamic / flexible schema

Horizontal scaling

Uses different query models (no standard language)

Stores data as JSON/BSON, documents, key-values, etc.

Best for big data and real-time applications

CAP theorem principles (high availability and scalability)

Examples: MongoDB, Cassandra, Redis, DynamoDB



Faster for large unstructured and semi-structured data

* Managing Collections

Collection:

- A collection in MongoDB is similar to a table in SQL.
- It holds multiple documents (records)
- These are containers for storing documents.

Managing Collections:

- It means performing various operations on collections.
- Managing collections means performing operations to create, read, update, delete collections.
- It means handling everything related to collections.

→ The operations are called CRUD operations.

→ CRUD stands for:

C - Create

R - Read

U - Update

D - Delete

⇒ It involves:

1. Creating Collections
2. Reading Collections
3. Update Collections
4. Delete Collections

5. Inserting documents into collections

6. Renaming collection → `renameCollection()`

7. Dropping Collections `drop()`

↓
delete entire collection from DB.

`insertOne()`
`insertMany()`

1. Creating Collection

→ We create collection by using createCollection()

Syn: db.createCollection("name")

name - table name.

Ex: db.createCollection("CSE")

2. Reading ~~data~~ collections

→ To display ~~documents~~ collection, we use find()

Syn: db.collection-name.find()

ex: db.CSE.find()

3. Updating

→ To modify the documents in collection, we use updateOne(), updateMany() or \$set

Syn: db.collection-name.update(criteria, update data)

ex: db.CSE.update({ "name": "AAA", \$set: { "name": "ABC" } })

4. Deleting

→ Removing data from collection.

→ We use deleteOne() or deleteMany()

Syn: db.collection-name.delete()

* MongoDB Data Types

⇒ MongoDB stores data in Documents.

⇒ Each document has different types of values.
- These are called Data types.

⇒ MongoDB supports many data types :

1. String
2. Number
3. Boolean
4. Array
5. Object
6. Null
7. Date
8. Double
9. Binary

1. String: used to store text. ~~used to store text.~~

- most common data-type.

ex: {
 "name": "ABC",
 "course": "B.tech",
 "Branch": "CSE"
}

(Int/Double) stores numbers
2. Number:

- Also called as Integer

ex: 22, 49.99

3. Boolean: stores true or false

Ex: "is Student": true.

4. Array: stores a list of values.

→ It can store any type of data.

Ex: "skills" : ["HTML", "CSS", "JS"]

5. Object: stores another document inside a document.

ex: "address" : {

"city" : "Hyd",

"pincode" : 500000

}

6. Null: stores a null or empty value.

ex: "middleName" : null

7. Date: stores date & time.

→ The current date or time is stored in the date datatype.

8. Binary Data: stores binary data
- like images or files

→ It is rarely used directly.

9. Code:

→ stores JavaScript code within documents.

→ used rarely.

• Double: stores floating point

49.9

• Integer: stores numbers

49

} Numbers

* Adding the MongoDB Driver to Node.js

→ Adding the MongoDB driver to Node.js means installing & using a sw Package (driver).

→ MongoDB Driver is a Node.js package.

→ Adding the MongoDB Driver to Node.js means

Setting up a connection b/w your Node.js application & MongoDB

- So they can work together

⇒ It means giving Node.js the ability to connect to MongoDB

⇒ It means installing a tool

- in your node.js project

- that helps it talk to the MongoDB

→ Just like you need a charger to connect your phone to power.

- You need a driver to connect Node.js to MongoDB.

~~⇒~~ ⇒ Node.js ~~are~~ does not understand how to talk to MongoDB directly.

- the driver acts like a translator b/w Node.js & MongoDB.

How to add ① install node.js (if not installed)

⇒ To add MongoDB driver in Node.js:

② We use this command to install the driver
`npm install mongodb`

③ Import the MongoDB driver
`const { MongoClient } = require('mongodb');`

④ You can now connect to the MongoDB database using the driver.

⑤ This allows you to insert, read, update, delete data from MongoDB using Node.js

⇒ It acts like bridge b/w Node.js & MongoDB

• once connected, we can perform CRUD operations

- Create (insert)
- Read (find) (display)
- Update
- Delete